SEMANA 15 · ARQUITECTURA EN LA NUBE

IA en la Nube II: GPU, Cuantización y Servicios Gestionados frente a Auto-Alojados

Parte 1 de 4 · Arquitectura de GPUs, Aceleradores y Jerarquía de Memoria

Ing. Rodolfo José Cañas Cervantes
Universidad de la Costa (CUC) · Ingeniería de Sistemas

Arquitectura en la Nube · Periodo 2026-2 · Barranquilla, Colombia

CPU frente a GPU en Cómputo de IA

De núcleos complejos optimizados para latencia secuencial a miles de ALUs paralelas SIMT

CPU Clásica (Latencia Secuencial)

Arquitectura: Pocos núcleos complejos
16 a 64 núcleos x86 de alta frecuencia (3.5 - 4.5 GHz)

Área de Silicio: Caché L3 y Branch Predictors
Gran parte del chip dedicada a predecir saltos y control

Rendimiento en IA: Cuello de botella serial
Incapaz de multiplicar matrices gigantes en microsegundos

Propósito: Lógica de negocio, sistemas operativos y APIs

GPU Moderna (Throughput Paralelo SIMT)

Arquitectura: Miles de ALUs diminutas
10,000 a 18,000 núcleos CUDA en ejecución simultánea

Área de Silicio: Unidades de Cómputo Puro
80 % del silicio dedicado a multiplicar y acumular (FMA)

Rendimiento en IA: Aceleración Matricial 100x
Multiplica billones de parámetros en paralelo continuo

Propósito: Inferencia y entrenamiento masivo

Fuente: NVIDIA CUDA Programming Guide & Computer Architecture: A Quantitative Approach

Ancho de Banda HBM3 y el Modelo Roofline

Diferenciación entre fases limitadas por cálculo (Prefill) y limitadas por memoria (Decode)

MODELO ROOFLINE: INTENSIDAD OPERACIONAL (FLOPs / Byte)



Fuente: Roofline: An Insightful Visual Performance Model for Multicore Architectures · LBNL / UC Berkeley

Tensor Cores y Formatos de Punto Flotante

De FP32 a FP8: optimización de representación de bits para duplicar el rendimiento matricial

FP32 (Precisión Simple)

32 bits: 1 signo, 8 exp, 23 mant

Rango y precisión astronómicos

Consumo: 4 bytes por parámetro

Un modelo 70B requeriría 280 GB

solo para alojar los pesos

Uso Actual en Nube:

Cálculo científico puro.
Obsoleto para inferencia LLM.

FP16 / BF16 (Media Precisión)

16 bits: signo, exp, mant

BF16 mantiene rango dinámico FP32

Consumo: 2 bytes por parámetro

Un modelo 70B consume 140 GB

Velocidad 2x frente a FP32

Uso Actual en Nube:

Estándar oro para entrenamiento
y pesos base originales.

FP8 (Transformer Engine)

8 bits: Formatos E4M3 y E5M2

Introducido en NVIDIA Hopper H100

Consumo: 1 byte por parámetro

Un modelo 70B cabe en 70 GB

Rendimiento 2x frente a FP16

Uso Actual en Nube:

Inferencia ultrarrápida y
reducción de KV-Cache.

INT4 (Cuantización Extrema)

4 bits: Enteros empaquetados

Algoritmos AWQ y GPTQ

Consumo: 0.5 bytes por parámetro

Un modelo 70B cabe en 35 GB

¡Cabe en 1 sola GPU de 48 GB!

Uso Actual en Nube:

Despliegue local y en el borde
con mínima pérdida de calidad.

Fuente: NVIDIA Hopper Architecture In-Depth · NVIDIA Developer Blog

Interconexiones: NVLink vs PCIe Gen 5

Superación de las limitaciones del bus de placa base mediante conmutación NVSwitch dedicada

PCIe Gen 5 x16 (Bus Tradicional)

Ancho de Banda Bidireccional: 64 GB/s
Límite físico del bus de interconexión con el procesador

Topología de Árbol (Root Complex)
El tráfico entre GPUs compite con la CPU y la memoria RAM

Impacto en Paralelismo Tensorial (TP)
Severo cuello de botella al sincronizar capas de modelos grandes

Solo inferencia independiente sin NVLink.

NVIDIA NVLink 4 + NVSwitch (Malla Dedicada)

Ancho de Banda Bidireccional: 900 GB/s
¡Más de 14 veces la velocidad de PCIe Gen 5!

Conmutación Any-to-Any Directa (NVSwitch)
Las 8 GPUs del servidor se comunican en una sola red unificada

Memoria Unificada Compartida (SHMEM)
Las GPUs leen la memoria de sus vecinas como si fuera local

Indispensable para Tensor Parallelism.

Fuente: NVIDIA NVLink & NVSwitch Architecture Specification · NVIDIA Corp.

Chips Propietarios: AWS Trainium e Inferentia

Alternativas de silicio diseñadas a medida para optimizar el costo por inferencia en modelos generativos

AWS Inferentia2 (Familias inf2)

Propósito: Inferencia de ultra bajo costo
Hasta 4x mayor throughput y 10x menor latencia vs inf1

Arquitectura NeuronCore-v2
Soporte nativo FP16, BF16, cfloat8 e INT8

Enlace Inter-Chip: NeuronLink-v2
384 GB/s en topología de anillo para modelos hasta 175B

Ahorro: hasta 40% menor costo que GPU.

AWS Trainium (Familias trn1 / trn1n)

Propósito: Entrenamiento e inferencia a escala masiva
Aceleración de LLMs y modelos de difusión generativa

Capacidad Hardware por Servidor trn1.32xlarge
16 chips Trainium (32 núcleos Neuron), 512 GB HBM2e

Red Ultra Rápida: 1600 Gbps EFA
Clústeres EC2 UltraClusters con decenas de miles de chips

Herramienta: AWS Neuron SDK (compilador para PyTorch)

Fuente: AWS Neuron SDK Documentation & Annapurna Labs Whitepaper

Familias de Instancias GPU en AWS EC2

Clasificación de tipos de instancias según perfil de carga, memoria de vídeo y presupuesto

Familia EC2 G5 (NVIDIA A10G)

Hardware de Aceleración:
1 a 8 GPUs NVIDIA A10G (24 GB GDDR6 c/u)
Hasta 192 GB VRAM agregada en g5.48xlarge

Costos y Eficiencia:
Desde $1.00 USD/hora (g5.xlarge: 1 GPU)
Máxima relación costo-beneficio para inferencia

Cargas Recomendadas:
- Modelos medianos (7B - 13B parámetros)
- Embeddings vectoriales y visión por computador

Estado: Muy popular para microservicios IA

Familia EC2 P4de (NVIDIA A100)

Hardware de Aceleración:
8 GPUs NVIDIA A100 (80 GB HBM2e c/u)
Total: 640 GB VRAM a 2.0 TB/s ancho de banda

Costos y Eficiencia:
Aprox. $40.96 USD/hora en bajo demanda
Requiere alta concurrencia para ser rentable

Cargas Recomendadas:
- Modelos grandes (70B sin cuantizar)
- Fine-tuning y entrenamiento de dominio

Estado: Caballo de batalla empresarial

Familia EC2 P5 (NVIDIA H100)

Hardware de Aceleración:
8 GPUs NVIDIA H100 SXM5 (80 GB HBM3 c/u)
3.35 TB/s de ancho de banda y 3200 Gbps EFA

Costos y Eficiencia:
Aprox. $98.32 USD/hora (p5.48xlarge)
Máxima gama de cómputo en la nube

Cargas Recomendadas:
- Modelos fundacionales (70B a 405B)
- Pre-entrenamiento y supercomputación

Estado: Capacidad de frontera en IA

Fuente: Amazon EC2 Instance Types Guide · AWS Documentation (2026)

Redes de Ultra-Baja Latencia: EFA y RoCEv2

Mecanismos de comunicación directa GPUDirect RDMA para evitar cuellos de botella de red en clústeres

Pila TCP/IP Clásica (Sobrecarga de CPU)

Múltiples Copias de Memoria:
VRAM GPU -> RAM Host -> Búfer Kernel -> Tarjeta NIC

Sobrecarga de Interrupciones en CPU
El procesador central gasta ciclos valiosos copiando paquetes

Latencia de Red Inestable (Jitter)
Latencias de 20 a 50 microsegundos con riesgo de pérdidas

Inviable para sincronización distribuida.

GPUDirect RDMA + AWS EFA (Zero-Copy)

Transferencia Directa de Memoria:
VRAM GPU Nodo 1 -> Red NIC -> VRAM GPU Nodo 2

Bypass Total del Sistema Operativo (OS Bypass)
La tarjeta de red accede a la memoria sin despertar a la CPU

Protocolo SRD (Scalable Reliable Datagram)
Enrutamiento multipath dinámico en la red física de AWS

Latencia sub-microsegundo para TP.

Fuente: AWS Elastic Fabric Adapter (EFA) Documentation & InfiniBand Trade Association

Particionamiento de GPU: NVIDIA MIG

Multi-Instance GPU para dividir físicamente una A100/H100 en hasta 7 instancias totalmente aisladas

GPU FÍSICA NVIDIA H100 (80 GB HBM3 / 114 SMs CÁLCULO)

MIG permite particionar hardware a nivel de silicio con canales de memoria y caché L2 dedicados

Instancia MIG 1g.10gb

1 Slice de Cómputo

- 10 GB HBM3 dedicados
- 14 Multiprocesadores SM
- Caché L2 independiente

Caso de Uso:
Embeddings vectoriales y clasificación ligera.

Instancia MIG 2g.20gb

2 Slices de Cómputo

- 20 GB HBM3 dedicados
- 28 Multiprocesadores SM
- Ancho de banda garantizado

Caso de Uso:
Inferencia de modelos 7B (Llama 3 8B en INT4).

Instancia MIG 3g.40gb

3 Slices de Cómputo

- 40 GB HBM3 dedicados
- 42 Multiprocesadores SM
- Alto ancho de banda VRAM

Caso de Uso:
Inferencia concurrente 13B y visión multimodal.

Ventaja Inmune a Ruidos

Aislamiento por Hardware

Cero interferencia ("Noisy Neighbor")

Si una partición sufre OOM, las demás 6 instancias siguen vivas.

Garantía de SLA:
Latencia predecible para múltiples departamentos.

Fuente: NVIDIA Multi-Instance GPU (MIG) User Guide · NVIDIA Docs

Eficiencia Energética y Refrigeración Líquida

De racks tradicionales de 10 kW a densidades de 40 a 100 kW por bastidor impulsadas por IA

Refrigeración Tradicional por Aire

Densidad Máxima de Rack: 10 a 15 kW
Límite térmico del flujo de aire frío impulsado por ventiladores

Consumo Energético Elevado (PUE 1.4 - 1.6)
Por cada watt de cómputo se gasta casi medio watt en climatización

Incompatibilidad con Servidores Modernos
Un solo chasis de 8 GPUs H100 consume 10.2 kW (inviable por aire)

Límite físico para centros de datos.

Refrigeración Líquida Direct-to-Chip (DLC)

Densidad Extrema de Rack: 40 a 100 kW
Placas frías de cobre en contacto directo con el chip de la GPU

Eficiencia Energética Excepcional (PUE 1.1)
El agua transporta 4,000 veces más calor que el aire por volumen

Sostenibilidad y Recuperación Térmica
El calor residual a 45 °C se reutiliza en calefacción o procesos

Obligatorio para nueva generación.

Fuente: Data Center Frontier AI Power Report & Open Compute Project (OCP) Guidelines

GPU Passthrough en Entornos OpenStack

Mapeo directo de dispositivos PCIe hacia máquinas virtuales KVM mediante IOMMU y VFIO

1. Configuración Host

Habilitación de IOMMU

En línea de comandos del kernel:

```
intel_iommu=on iommu=pt
```

Aislamiento de bus PCIe

Desvincula el driver nouveau

y asocia el driver vfio-pci

al ID del fabricante NVIDIA

2. Nova Placement

Parámetros nova.conf

Declara el alias del dispositivo:

```
[pci] device_spec =
{"vendor_id": "10de",
"product_id": "2235"}
```

Define el alias en scheduler:

```
alias = {"name": "gpu-a10"}
```

3. Creación de Flavor

Propiedad Extra Spec

Comando del administrador:

```
openstack flavor set \
--property pci_passthrough:\
alias='gpu-a10:1' m1.gpu
```

El scheduler filtra los nodos

que tengan tarjetas libres

4. Máquina Virtual

Acceso Nativo al Hardware

La VM detecta el dispositivo:

```
lspci | grep -i nvidia
NVIDIA A10G (rev a1)
```

Instalación normal de CUDA

Cero penalización de emulación

Rendimiento 100 % nativo

Fuente: OpenStack Nova PCI Passthrough Guide (docs.openstack.org/nova) · OpenInfra

TFLOPS, Ancho de Banda y MFU

Indicadores fundamentales para cuantificar la eficiencia de ejecución en aceleradores de IA

1. TFLOPS Brutos de Cómputo

Capacidad Teórica Pico:

Billones de operaciones de coma flotante por segundo de los Tensor Cores.

Dependencia de la Precisión:

H100 entrega 1,000 TFLOPS en FP16

y escala a 2,000 TFLOPS en precisión FP8

Relevancia Práctica:

Determina el tiempo de procesamiento del prompt inicial en la fase de prefill.

2. Ancho de Banda VRAM (TB/s)

Tasa de Transferencia de Datos:

Velocidad a la que el chip puede leer pesos y la memoria caché desde la VRAM

Comparativa Tecnológica:

A10G (GDDR6): 0.60 TB/s

H100 (HBM3): 3.35 TB/s (5.5x más rápido)

Relevancia Práctica:

Dicta la latencia por token de salida en la fase de decode de los chatbots.

3. MFU (Model Flops Utilization)

Porcentaje de Eficiencia Real:

MFU = FLOPs ejecutados útiles / Capacidad teórica del acelerador

Valores Observados en Producción:

Motores no optimizados: 15 % a 25 % MFU

vLLM con PagedAttention: 45 % a 60 % MFU

Relevancia Práctica:

Mide si el software está aprovechando verdaderamente el hardware pagado.

Fuente: Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism

Monitorización de GPUs con NVIDIA DCGM

Telemetría Prometheus para detectar sobrecalentamiento, estrangulamiento térmico y errores de bus

1. Utilización y Temperatura

Métrica: DCGM_FI_DEV_GPU_UTIL
Porcentaje de uso de núcleos de cómputo
Debe ser continuo en cargas de producción

Métrica: DCGM_FI_DEV_GPU_TEMP
Temperatura del núcleo de silicio (°C)
Alerta si supera los 83 °C sostenidos

Métrica: Thermal Throttling
DCGM_THROTTLE_REASONS
Indica si la GPU baja MHz por calor

Alerta de refrigeración en datacenter

2. Memoria y OOM Risks

Métrica: DCGM_FI_DEV_FB_USED
Megabytes de VRAM en uso
Suma de pesos, activaciones y KV-Cache

Métrica: DCGM_FI_DEV_FB_FREE
Megabytes libres restantes en VRAM
Alerta roja si margen libre cae de 1 GB

Métrica: Errores ECC de Memoria
DCGM_FI_DEV_ECC_SBE_VOL_TOTAL
Single/Double bit errors en HBM3

Prevención de fallo de módulo VRAM

3. Interconexión y Bus PCIe

Métrica: DCGM_FI_DEV_PCIE_TX_BYTES
Tráfico de transmisión por PCIe
Bytes transferidos host-to-device

Métrica: NVLINK_BANDWIDTH
Throughput agregado de NVLink
Detecta enlaces degradados en malla

Métrica: XID Driver Errors
Eventos de pánico del kernel de GPU
XID 31 (falla memoria), XID 79 (caída)

Herramienta: dcgm-exporter en K8s

Fuente: NVIDIA Data Center GPU Manager (DCGM) Documentation

Selección de GPUs en la CIC Barranquilla

Análisis de dimensionamiento para el asistente industrial de planta química en la Vía 40

Opción 1: Instancia AWS P4d

Hardware Evaluado:

8x NVIDIA A100 (320 GB VRAM)
Costo: ~$32.77 USD por hora

Factura Mensual 24/7:

Aprox. $23,594 USD / mes

Gasto desproporcionado para inferencia

Veredicto Corporativo:

Sobredimensionado. Inviable para el presupuesto operativo de planta.

Opción 2: Instancia AWS G5

Hardware Evaluado:

g5.2xlarge: 1x NVIDIA A10G (24 GB)
Costo: ~$1.21 USD por hora

Factura Mensual 24/7:

Aprox. $871 USD / mes

Excelente para pruebas cloud

Veredicto Corporativo:

Aceptable en costo pero sufre latencia WAN hacia el SCADA local (120 ms).

Opción 3: Servidor On-Premise

Hardware Seleccionado:

2x NVIDIA RTX 6000 Ada (48 GB c/u)
Inversión inicial CapEx: ~$18,000 USD

Amortización a 3 Años:

Aprox. $500 USD / mes (con energía)

Latencia ultra-baja en red local: < 5 ms

Veredicto Ganador:

Óptimo para planta química: costo predecible y datos en sitio seguro.

Principios Clave de Cómputo Acelerado

Pautas de diseño para seleccionar, operar y optimizar plataformas de GPU en producción

Reglas 1 a 5: Hardware y Memoria

1. Priorizar Ancho de Banda HBM sobre TFLOPS
En inferencia conversacional, la memoria VRAM dicta el throughput final.

2. Adoptar Precisión Reducida por Defecto
Utilizar FP16, BF16 o FP8 en Tensor Cores para duplicar velocidad de cálculo.

3. Interconexión NVLink Obligatoria para TP
No desplegar TP sobre PCIe tradicional;

4. Particionar con MIG en Cargas Heterogéneas
Dividir tarjetas A100/H100 para garantizar SLAs independientes sin interferencia.

5. Monitoreo DCGM de Errores Térmicos y ECC
Supervisar throttling térmico y códigos XID para prevenir caídas de nodo.

Reglas 6 a 10: Redes y Gobernanza

6. Redes RDMA (EFA / RoCEv2) en Clústeres
Bypass del sistema operativo para sincronizaciones colectivas sin latencia.

7. Considerar Chips Propietarios (Inferentia)
Evaluar instancias inf2 para reducir la factura mensual en modelos Transformer.

8. Refrigeración Líquida en Densidades > 30 kW
Planificar infraestructura Direct-to-Chip para servidores de última generación.

9. GPU Passthrough Directo en Nube Privada
Asignar dispositivos PCIe nativos con VFIO en KVM para eliminar sobrecarga.

10. Medir Eficiencia con MFU Continuo
Auditar porcentaje de Tensor Cores usados;

Fuente: AI Infrastructure Architecture Guidelines · Stanford HAI / ACM

SEMANA 15 · ARQUITECTURA EN LA NUBE

IA en la Nube II: GPU, Cuantización y Servicios Gestionados frente a Auto-Alojados

Parte 2 de 4 · Técnicas de Optimización, Compresión y Cuantización

Ing. Rodolfo José Cañas Cervantes
Universidad de la Costa (CUC) · Ingeniería de Sistemas

Arquitectura en la Nube · Periodo 2026-2 · Barranquilla, Colombia

Presupuesto de VRAM en Inferencia

Desglose estricto entre pesos del modelo, activaciones transitorias y memoria de contexto KV-Cache

1. Pesos del Modelo (Estático)

Fórmula de Ocupación Base:

Memoria = Parámetros × Bytes/peso
Constante durante toda la ejecución

Ejemplo en Modelo 70B:

- FP16 (2 bytes): 140 GB VRAM
- INT4 (0.5 bytes): 35 GB VRAM

Impacto en Servidor:

Define el número mínimo de GPUs para poder arrancar el servicio.

2. KV-Cache (Dinámico)

Fórmula de Contexto:

2 × Capas × Cabezas × Dim × Long
Crece linealmente con cada token

Escalado por Concurrencia:

Multiplica por el número de usuarios
100 sesiones concurrentes = 20-50 GB

El Peligro del OOM:

Prompts muy largos saturan VRAM provocando caída por falta de memoria.

3. Activaciones y Búferes

Espacio de Cómputo Scratch:

Tensores intermedios en prefill y búferes temporales de CUDA

Sobrecarga Típica:

1 a 3 GB adicionales por instancia
Baja huella comparada con la KV-Cache

Regla de Dimensionamiento:

Reservar al menos 20 % de VRAM libre para crecimiento dinámico de sesión.

Fuente: Transformer Inference Memory Breakdown · Hugging Face Technical Blog

Cuantización: Mapeo de FP16 a INT8 e INT4

Transformación lineal afín para comprimir tensores continuos en cuadrículas discretas de enteros

FÓRMULA GENERAL DE CUANTIZACIÓN LINEAL AFÍN (Asimétrica y Simétrica)

$$q = clip(round(x / S) + Z, q_min, q_max)$$

Donde: x = tensor FP16 original · S = Factor de Escala (Scale) · Z = Punto Cero (Zero-Point) · q = entero cuantizado

Cuantización Simétrica (Zero-Point Z = 0)

Mapeo Centrado en Cero:

Rango: [-q_max, +q_max] · S = max(|x|) / q_max

Ventaja en Hardware:

Las multiplicaciones matriciales no requieren sumar el término Z, ejecutando a máxima velocidad nativa en los Tensor Cores.

Cuantización Asimétrica (Zero-Point Z ≠ 0)

Mapeo con Desplazamiento:

Rango: [x_min, x_max] · S = (x_max - x_min) / (q_max - q_min)

Aplicación Práctica:

Ideal para funciones de activación no simétricas como ReLU o GeLU, donde los valores son predominantemente positivos.

Fuente: Quantization and Training of Low Precision Neural Networks · Google Research

PTQ frente a QAT

Cuantización post-entrenamiento rápida frente a ajuste fino con simulación de pérdida de precisión

PTQ: Post-Training Quantization (Estándar)

Proceso: Conversión sobre pesos ya entrenados
No requiere reentrenar ni disponer del dataset original

Tiempo de Cómputo: Minutos a pocas horas
Solo requiere unas pocas muestras de calibración (ej. 128)

Algoritmos Modernos: AWQ, GPTQ, GGUF
Pérdida de perplejidad casi nula en modelos > 13B

Elección universal para adopción corporativa.

QAT: Quantization-Aware Training

Proceso: Simula cuantización durante el forward pass
Los gradientes en backward pass se calculan en FP32/BF16

Tiempo de Cómputo: Muy costoso (Días/Semanas)
Demanda clústeres masivos de GPUs y datasets masivos

Recuperación de Calidad: Máxima posible
La red aprende a tolerar el redondeo adaptando los pesos

Para fabricantes de chips y clústeres.

Fuente: A Survey of Quantization Methods for Efficient Neural Network Inference · IEEE Proceedings

Algoritmo GPTQ: Compensación del Hessiano

Minimización analítica del error cuadrático entre capas mediante la inversa de la matriz Hessiana

1. Matriz Hessiana Inversa (H^-1)

Cálculo de Curvatura de Error:

H = 2 * X * X^T a partir de activaciones

Propósito Matemático:

Determina qué pesos son más sensibles y cómo se propaga el error de redondeo

Inversión por Cholesky:

Estabilidad numérica optimizada para matrices de 4096 dimensiones.

2. Cuantización y Ajuste Lazy

Redondeo de Columna i:

q_i = quant(w_i) a 4 bits enteros

Compensación Inmediata:

```
w_restantes -= (w_i - q_i) * (H^-1)_i
```

Los pesos no cuantizados absorben el error

Actualización en Bloques:

Agrupa 128 o 256 columnas para aprovechar la memoria GPU al máximo.

3. Empaquetado en VRAM

Empaquetado de Bits:

8 enteros de 4-bits en 1 entero INT32

Descompresión al Vuelo (De-quant):

Kernel CUDA descomprime a registros

Cero ancho de banda extra consumido

Resultado Operativo:

Modelo 70B corre en 2 GPUs A10G con throughput 3x superior a FP16.

Fuente: GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers · Frantar et al.

Algoritmo AWQ: Preservación de Outliers

Identificación de los canales críticos de activación para proteger el 1 % de pesos más influyentes

Cuantización Ingenua (Todos por Igual)

El Fenómeno de los Canales Atípicos (Outliers):
El 1 % de los canales presenta activaciones 100x más grandes

Error Catastrófico de Redondeo
Cuantizar los canales críticos destruye la atención del modelo

Impacto en Razonamiento y Código
Caída severa de precisión en benchmarks matemáticos y Python

Falla crítica en tareas que requieren precisión lógica estricta

AWQ: Activation-aware Weight Quantization

Protección del 1 % de Pesos Salientes:
Identifica qué pesos interactúan con activaciones de alta magnitud

Escalado Matemático por Canal (Per-Channel Scale):
Multiplica los pesos salientes por factor s y divide activaciones por s

Mantiene Formato Puro INT4:
Sin necesidad de formatos mixtos complejos en el hardware

Calidad idéntica a FP16 en silicio.

Fuente: AWQ: Activation-aware Weight Quantization for On-Device LLM Compression · Lin et al. (MIT)

FORMATO UNIVERSAL HÍBRIDO

GGUF y Llama.cpp: Inferencia en CPU y GPU

Contenedor binario autocontenido para despliegues ligeros y descarga parcial de capas (Offloading)

Estructura del Formato Binario GGUF

Archivo Único Autocontenido (.gguf):
Incluye arquitectura, vocabulario del tokenizador y tensores

Mapeo Directo en Memoria (mmap):
Arranque instantáneo en segundos sin cargar todo a RAM

Cuantizaciones Mixtas (K-Quants):
Q4_K_M: Pesos críticos en 5 o 6 bits, resto en 4 bits

Portabilidad: Linux, macOS y Windows.

Descarga Híbrida de Capas (GPU Offload)

Capas en VRAM de la GPU (-ngl 24):
Las primeras 24 capas del modelo se descargan a la VRAM
Aceleración masiva por hardware en el espacio disponible

Capas Restantes en RAM del Sistema:
Las capas sobrantes ejecutan en CPU con instrucciones AVX-512
Permite correr modelos de 30 GB en servidores con solo 16 GB de GPU

Caso de Uso Empresarial:

- Estaciones de trabajo de ingeniería y desarrollo local
- Servidores perimetrales edge sin presupuesto de datacenter
- **Despliegue de Ollama en máquinas virtuales departamentales**

Fuente: GGUF Specification & Llama.cpp Architecture Documentation · Georgi Gerganov

Métricas de Degradación: Perplejidad y MMLU

Cuantificación objetiva del impacto de la compresión en la coherencia lingüística y razonamiento

Perplejidad (Perplexity - PPL)

Definición Matemática:
Exponencial de la entropía cruzada media por token

Interpretación Intuitiva:
A menor perplejidad, más seguro y certero es el modelo

Impacto Típico de Cuantización (Llama 3 70B):
FP16: PPL 3.12 · AWQ INT4: PPL 3.18 (+1.9 % apenas)

Métrica continua sensible a degradación.

Evaluación en Tareas Reales (Downstream)

Benchmark MMLU (Conocimiento Multidisciplinar):
FP16: 79.5 % de acierto · INT4 AWQ: 78.9 % de acierto

Benchmark HumanEval (Generación de Código):
FP16: 62.2 % pass@1 · INT4 AWQ: 61.4 % pass@1

Benchmark GSM8K (Razonamiento Matemático):
Evaluación de resolución lógica paso a paso

Pérdida < 1% con reducción del 75%.

Fuente: Evaluating Quantized Large Language Models · EleutherAI Benchmarking Suite

Poda (Pruning) y Dispersión Estructurada 2:4

Aceleración de matrices dispersas soportada por hardware nativo en los núcleos NVIDIA Ampere y Hopper

Poda No Estructurada (Dispersión Aleatoria)

Concepto: Poner a cero los pesos más pequeños

Si un peso es muy cercano a 0.00, se elimina

Incompatibilidad con Hardware Paralelo

Las matrices dispersas requieren índices CSR lentos de leer

Paradoja de Rendimiento:

Pese a tener 50 % menos datos, la GPU ejecuta más lento

Sin beneficio en inferencia moderna.

Dispersión Estructurada 2:4 (NVIDIA Sparse)

Regla Estricta: Exactamente 2 ceros por cada 4 pesos

Patrón geométrico predecible y alineado a nivel de hardware

Aceleración Física en Tensor Cores 2x:

El hardware omite los ciclos de multiplicación por cero

Compresión de Almacenamiento al 50 %:

Solo se guardan 2 valores y un mapa de 2 bits de índices

Duplica rendimiento matemático.

Fuente: Accelerating Sparse Deep Neural Networks with NVIDIA Ampere Architecture and Tensor Cores

Destilación de Conocimiento (Distillation)

Transferencia de capacidades desde un modelo maestro gigante hacia un modelo alumno compacto y veloz

Modelo Maestro (Teacher - 70B / 405B)

Capacidad Cognitiva Sobresaliente:

Razonamiento profundo, síntesis y generación lógica

Distribución Suave de Probabilidades (Soft Targets):

Emite las probabilidades relativas de todos los tokens alternativos

La "Materia Oscura" del Aprendizaje:

El alumno no solo aprende cuál es la respuesta correcta, sino qué respuestas son casi correctas y cuáles son absurdas.

Modelo Alumno (Student - 8B / 1B)

Arquitectura Ligera y Ágil:

10x a 20x menor consumo de memoria y computación

Entrenamiento Guiado por KL-Divergence:

Ajusta sus pesos para imitar la distribución suave del maestro

Resultado Sobresaliente:

Un modelo de 8B destilado puede superar en tareas específicas a modelos de 70B entrenados desde cero.

Fuente: Distilling the Knowledge in a Neural Network · Geoffrey Hinton et al.

Evolución de la Atención: MHA, MQA y GQA

Reducción radical de cabezas de clave y valor para recortar hasta un 87 % del consumo de KV-Cache

MHA (Multi-Head Attention)

Estructura Tradicional:
32 Cabezas Query, 32 Key, 32 Value
Relación 1:1 estricta en cada capa

Consumo de KV-Cache: 100 %
Cada token añade 32 vectores K y V
Gigantesca huella de memoria en VRAM

Limitación Operativa:
Imposible escalar a contextos largos
de 32k o 128k con múltiples usuarios.

Modelos: GPT-3, Llama 1 original.

MQA (Multi-Query Attention)

Estructura Extrema:
32 Cabezas Query, 1 Key, 1 Value
Todas las cabezas comparten 1 solo par

Consumo de KV-Cache: 3.1 %
¡Reducción masiva del 96.9 % en VRAM!
Velocidad de generación supersónica

Compromiso de Calidad:
Puede perder riqueza semántica
en tareas complejas de múltiples giros.

Modelos: PaLM, Falcon 7B.

GQA (Grouped-Query Attention)

El Equilibrio Perfecto:
32 Cabezas Query agrupadas en 8 grupos
4 Queries comparten 1 cabeza Key y Value

Consumo de KV-Cache: 25 %
Ahorra el 75 % de VRAM frente a MHA
Calidad indistinguible de MHA puro

Estándar de la Industria:
Permite contextos de 128k tokens
sin desbordar la memoria de la GPU.

Modelos: Llama 2 (70B), Llama 3, Mistral.

Fuente: GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints

Cuantización de la KV-Cache en FP8 e INT8

Almacenamiento de claves y valores pasados en 8 bits para duplicar la concurrencia de usuarios

KV-Cache Estándar en FP16 (2 Bytes/token)

Consumo por Sesión (Contexto 8k tokens):
Aprox. 1.05 GB de VRAM por usuario conectado en Llama 70B

Concurrencia Máxima en GPU de 24 GB:
Tras cargar pesos cuantizados, solo caben 4 a 6 usuarios

Cuello de Botella en Throughput:
El servidor debe rechazar peticiones o encolar usuarios

Riesgo de saturación bajo concurrencia.

KV-Cache Cuantizada en FP8 (1 Byte/token)

Consumo por Sesión (Contexto 8k tokens):
Aprox. 0.52 GB de VRAM por usuario (50 % de reducción)

Concurrencia Duplicada:
La misma GPU soporta de 10 a 14 usuarios simultáneos

Soporte Nativo en vLLM:
Parámetro: --kv-cache-dtype fp8 (sin reentrenamiento)

Duplica capacidad operativa sin hardware adicional

Fuente: vLLM KV Cache Quantization Guide (docs.vllm.ai) · vLLM Project

Compiladores de Inferencia: TensorRT-LLM

Fusión de kernels CUDA, eliminación de transferencias intermedias y optimizaciones deep-learning

1. Grafo PyTorch Nativo

Operaciones Aisladas

Cada capa ejecuta un kernel:

```
Linear -> VRAM
RMSNorm -> VRAM
SiLU -> VRAM
```

Sobrecarga masiva de lectura y escritura de tensores en VRAM

Lanza miles de llamadas al driver

2. Fusión de Kernels

Optimización de Grafo

TensorRT compila y fusiona:

```
Linear + RMSNorm + SiLU
```

¡En un solo kernel CUDA!

Los datos intermedios quedan en registros ultra veloces

Cero viajes innecesarios a VRAM

3. FlashAttention Nativo

Cálculo de Atención Tiled

Algoritmo FlashAttention-2/3:

```
SRAM Tiling por Bloques
```

Sin materializar matriz N x N

Calcula softmax por partes

Acelera la atención entre 2x y 4x

Fundamental para contextos largos

4. Motor Compilado

Plan de Ejecución Binario

Archivo .engine específico

Rendimiento Máximo:

Latencia reducida a la mitad

Optimizaciones a medida para la GPU exacta instalada

Throughput industrial pico

Fuente: NVIDIA TensorRT-LLM Architecture Overview & Benchmarks · NVIDIA Developer

Compresión de Llama 70B para Servidor Único

De 140 GB en FP16 a 38 GB en AWQ INT4 para ejecutar en un único servidor con 2 GPUs A10G

Modelo Original Llama 3 70B (FP16)

Peso en Memoria VRAM: 140 GB

70,000 millones de parámetros × 2 bytes por peso

Requisito de Hardware Mínimo:

2x GPUs A100 (80 GB) o 8x GPUs A10G (24 GB)

Costo Operativo en la Nube:

Instancia g5.48xlarge o p4de: > $8,000 a $20,000 USD/mes

Inviable para proyectos medianos con presupuesto limitado

Modelo Optimizado con AWQ INT4 + FP8 KV

Peso en Memoria VRAM: 38 GB

35 GB de pesos cuantizados + 3 GB búferes de activación

Requisito de Hardware Reducido:

2x GPUs A10G (24 GB) o 1x RTX 6000 Ada (48 GB)

Costo Operativo en la Nube:

Instancia g5.12xlarge: ~$1,800 USD/mes (Ahorro del 77 %)

Retención de calidad superior al 99%.

Comparativa de Métodos de Cuantización

Criterios para seleccionar el algoritmo y formato ideal según el entorno de despliegue

CUADRO COMPARATIVO: FORMATOS DE CUANTIZACIÓN PARA PRODUCCIÓN

Formato / Método	Precisión Bits	Hardware Destino	Pérdida Calidad	Caso de Uso Recomendado
AWQ (Activation-aware) Protección 1% outliers	**4-bits enteros**	GPUs NVIDIA (vLLM / TGI)	< 0.8 % en MMLU	**Servidores Cloud de alta** concurrencia y microservicios.
GPTQ (Second-Order) Compensación Hessiano	**4-bits / 8-bits**	GPUs NVIDIA y AMD	< 1.2 % en MMLU	Auto-alojamiento estándar en entornos Linux y K8s.
GGUF (Llama.cpp) K-Quants híbridos	**2 a 8-bits mixtos**	CPU pura + GPU Offload	< 1.0 % (en Q4_K_M)	**Dispositivos edge, Ollama y** estaciones de desarrollo local.
FP8 (Transformer Engine) Formato de punto flotante	**8-bits nativos**	NVIDIA Ada, Hopper, Blackwell	Prácticamente 0.0 %	**Entrenamiento e inferencia** de modelos de frontera.

Fuente: LLM Quantization Survey & Framework Benchmarks (2026)

SEMANA 15 · ARQUITECTURA EN LA NUBE

IA en la Nube II: GPU, Cuantización y Servicios Gestionados frente a Auto-Alojados

Parte 3 de 4 · Motores de Inferencia de Alto Rendimiento Auto-Alojados

Ing. Rodolfo José Cañas Cervantes
Universidad de la Costa (CUC) · Ingeniería de Sistemas

Arquitectura en la Nube · Periodo 2026-2 · Barranquilla, Colombia

Anatomía de un Servidor de Inferencia

Desacoplamiento entre el frontend HTTP/gRPC, el planificador de lotes y el motor de ejecución CUDA

1. Servidor API y Encolado

Compatibilidad OpenAI API:

Endpoints: /v1/chat/completions y /v1/embeddings

Tokenización y Validación:

Convierte cadenas de texto en token IDs
Valida longitud de contexto permitida

Servidor Asíncrono FastAPI:

Streaming vía Server-Sent Events (SSE)
para entregar tokens conforme se generan.

2. Planificador (Scheduler)

Continuous Batching:

Agrupa peticiones a nivel de iteración de token

Gestor de Memoria de Bloques:

Monitorea bloques libres de KV-Cache en VRAM
Si la VRAM escasea, preempta peticiones a CPU

Priorización de Tráfico:

Intercala prefill de prompts entrantes con
el decode de las conversaciones en curso.

3. Motor de GPU (Worker)

Kernels CUDA Optimizados:

Ejecución de FlashAttention y GEMM AWQ

Tensor Parallelism (NCCL):

Sincronización AllReduce entre GPUs locales
Distribuye matrices en paralelo por NVLink

Retorno de Logits y Muestreo:

Aplica temperature, top-p y rep_penalty
entregando los nuevos tokens al cliente.

Fuente: vLLM Engine Architecture & Fast Distributed Inference Guide · vLLM Team

vLLM y PagedAttention: Cero Fragmentación

Inspirado en la memoria virtual de los sistemas operativos para eliminar el 80 % de desperdicio en VRAM

Asignación Tradicional (Memoria Contigua)

Reserva Estática por Longitud Máxima (max_len):
Reserva memoria para 4k o 8k tokens antes de generar

Fragmentación Interna Brutal (60 % - 80 %):
Si el usuario pide 50 tokens, el resto reservado se desperdicia

Fragmentación Externa entre Peticiones:
Huecos no contiguos que la GPU no puede asignar

Limita severamente el batch size y la concurrencia

PagedAttention (Memoria Paginada vLLM)

División en Bloques Físicos Discretos:
Cada bloque aloja exactamente 16 tokens de KV-Cache

Tabla de Páginas Lógicas a Físicas:
Los bloques no necesitan ser contiguos en la memoria de la GPU

Desperdicio de VRAM Menor al 4 %:
Solo se asigna un bloque nuevo cuando el anterior se llena

Multiplica el throughput del servidor entre 2x y 4x

Fuente: vLLM: Efficient Memory Management with PagedAttention · Kwon et al. (UC Berkeley)

Continuous Batching (In-Flight Batching)

Planificación dinámica a nivel de iteración de token para erradicar las colas de espera en inferencia

Batching Estático Tradicional (Naive)

Lote Bloqueado hasta Completitud:
Un lote de 8 peticiones corre junto hasta que la más larga termine

Relleno Inútil (Padding Overhead):
Las secuencias cortas rellenan ceros y esperan a la más lenta

Latencia de Cola Disparada:
Nuevas peticiones esperan bloqueadas en cola fuera del lote

Desperdicio severo de ciclos GPU y latencias P99 elevadas

Continuous Batching (Orca / vLLM)

Planificación a Nivel de Token (Iteration-Level):
El lote se reconfigura dinámicamente en cada ciclo de avance

Inserción Inmediata de Nuevas Peticiones:
Cuando una secuencia emite <eos>, otra entra sin esperar

Cero Relleno de Padding:
Todas las operaciones de cálculo corresponden a tokens reales

Rendimiento entre 10x y 20x superior frente a lotes estáticos

Fuente: Orca: A Distributed Serving System for Transformer-Based Generative Models · OSDI

Decodificación Especulativa (Speculative Decoding)

Generación tentativa de tokens con modelo borrador ligero y validación paralela en el modelo principal

1. Modelo Borrador (Draft - 1B / 8B)

Generación Rápida y Tentativa:

Genera especulativamente K tokens (ej. K = 4 o 5 tokens)

Bajo Costo Computacional:

Ejecuta en milisegundos por su reducido tamaño de memoria

Secuencia Tentativa Producida:

```
[ "El", "proceso", "de", "producción", "química" ]
```

Se remite en bloque al modelo maestro para verificación.

2. Modelo Objetivo (Target - 70B)

Verificación Paralela en 1 Solo Forward:

Evalúa los K tokens simultáneamente como un prompt (Prefill)

Aceptación o Corrección Matemática:

Acepta los tokens válidos y corrige el primero divergente

Aceleración Global Sin Pérdida:

Produce de 2 a 3 tokens por ciclo de GPU
¡Distribución matemática idéntica a correr solo el 70B!

Fuente: Fast Inference from Transformers via Speculative Decoding · Leviathan et al. (Google Research)

vLLM frente a TGI y Triton

Comparativa de arquitecturas de software para servicio de modelos en producción

vLLM (UC Berkeley / OpenInfra)

Puntos Fuertes:
- Creador nativo de PagedAttention
- Máximo throughput en alta concurrencia

Formatos Soportados:
AWQ, GPTQ, FP8, bitsandbytes
Soporte multi-GPU por Tensor Parallelism

Ecosistema y Facilidad:
- Compatible 100 % con OpenAI API
- Despliegue en 1 sola línea de comando

Recomendación: Estándar para LLMs puros

Hugging Face TGI (Text Gen)

Puntos Fuertes:
- Servidor escrito en Rust de alta fiabilidad
- Integración nativa con Hub de Hugging Face

Formatos Soportados:
AWQ, GPTQ, EETQ, Marlin
Soporte de FlashAttention-2 integrado

Ecosistema y Seguridad:
- Excelente telemetría OpenTelemetry
- Límites de tokens y watermarking nativo

Para entornos empresariales estrictos.

NVIDIA Triton Inference Server

Puntos Fuertes:
- Servidor multimodal universal
- Soporta LLMs, visión, audio y tabular

Backends Soportados:
TensorRT-LLM, ONNX, PyTorch, vLLM
Máxima optimización de hardware NVIDIA

Ecosistema y Complejidad:
- Curva de configuración más empinada
- Ensembles complejos de modelos encadenados

Para pipelines heterogéneos complejos.

Fuente: Open Source LLM Serving Frameworks Benchmark · Anyscale / Databricks

Despliegue en Kubernetes y KEDA

Escalado horizontal dinámico de pods de GPU gobernado por métricas de cola y tokens pendientes

ORQUESTACIÓN DE INFERENCIA EN KUBERNETES (KServe / vLLM Operator)

Autoescalado horizontal basado en métricas de negocio reales mediante KEDA (Kubernetes Event-driven Autoscaling)

1. Métricas de Cola Prometheus

Métricas Nativas Expuestas:

- vllm:num_requests_waiting
- vllm:num_requests_running
- vllm:gpu_cache_usage_factor

Por qué NO usar CPU/RAM:

La GPU siempre reporta 100 % de uso aunque esté procesando una sola petición.

2. ScaledObject de KEDA

Regla Declarativa en YAML:

```
triggers:
- type: prometheus
threshold: '5' # 5 en espera
```

Acción Automática:

KEDA escala el Deployment de vLLM desde 1 pod hacia N pods en paralelo.

3. Aprovisionamiento Karpenter

Autoescalado de Nodos Físicos:

Detecta pods pendientes por falta de GPU
Lanza instancias EC2 g5.12xlarge en < 2 min

Consolidación y Escala a Cero:

Drena y apaga servidores ociosos ahorrando costos durante la noche.

Fuente: Kubernetes Event-driven Autoscaling (KEDA) & vLLM Production Helm Guide

Paralelismo Tensorial y de Pipeline

Técnicas de partición de capas matriciales entre múltiples tarjetas aceleradoras mediante NCCL

Tensor Parallelism (TP - Intra-Nodo)

Divide matrices de pesos en un nodo.
Column Parallel en proyección inicial, Row Parallel en salida

Requisito de Red: Ultra Alta Velocidad (NVLink)
Requiere operaciones colectivas AllReduce en cada capa

Impacto en Latencia: Mínima (Ideal para inferencia)
Reduce la latencia por token al acelerar el cálculo en paralelo

Solo usar TP en el mismo nodo físico.

Pipeline Parallelism (PP - Inter-Nodo)

Mecanismo: Divide capas completas entre servidores
Nodo 1 aloja capas 1-16; Nodo 2 aloja capas 17-32

Requisito de Red: Redes Clúster EFA / RoCEv2
Solo transmite activaciones intermedias al final de cada bloque

El Problema de la Burbuja (Pipeline Bubble):
Incrementa la latencia por token al encadenar etapas

Usar PP si el modelo no cabe en un nodo.

Fuente: Megatron-LM: Training and Inference at Scale · Shoeybi et al. (NVIDIA)

MÉTRICAS CLAVE DE INFERENCIA

TTFT, TPOT y Throughput

Indicadores de calidad de servicio (SLA) para cuantificar la experiencia de usuario y la capacidad del sistema

1. TTFT (Time To First Token)

Definición de Latencia Inicial:
Tiempo desde que el usuario envía el prompt
hasta que ve el primer carácter en pantalla

Fase Asociada: Prefill
Procesamiento paralelo de la entrada
Depende de los TFLOPS brutos y longitud

Umbral de SLA Humano:
Excelente: < 200 ms · Aceptable: < 800 ms
> 1.5s genera sensación de lentitud severa

Impacto: Percepción de agilidad

2. TPOT (Time Per Output Token)

Definición de Fluidez:
Tiempo medio entre cada token generado
Equivalente inverso a la velocidad de lectura

Fase Asociada: Decode
Generación secuencial autorregresiva
Depende del ancho de banda de memoria VRAM

Umbral de SLA Humano:
Excelente: < 25 ms/tok (40 tokens/s)
Velocidad de lectura humana: 5-8 tok/s

Impacto: Fluidez de lectura natural

3. Throughput del Sistema

Capacidad Global Agregada:
Tokens totales procesados por segundo
Suma de todos los usuarios concurrentes

Métrica de Costo-Eficiencia:
Tokens / Segundo / Dólar de hardware
Escala linealmente con continuous batching

Objetivo en Servidor vLLM:
Superar los 1,500 - 3,000 tokens/s
por nodo con 4x GPUs A10G en batch alto

Impacto: Rentabilidad y escala máxima

Fuente: LLM Inference Performance Metrics Standard · MLPerf Inference Consortium

Control de Latencia P99: Chunked Prefill

Fragmentación de prompts gigantescos para evitar que monopolicen los ciclos de GPU y congelen el decode

Prefill Monolítico (Sin Chunking)

Llegada de un Documento de 16k Tokens:
Un usuario envía un PDF largo para resumir

Monopolio Total de la GPU:
La GPU gasta 1,200 ms continuos computando el prompt

Congelación de los Otros Usuarios (Jitter):
Las conversaciones activas sufren una pausa de más de 1 segundo

Pico inaceptable en la latencia percentil 99 (P99 Spike)

Chunked Prefill (Sarathi / vLLM v0.6+)

División en Fragmentos (ej. 512 Tokens):
El prompt largo se divide en trozos manejables

Intercalado Simétrico de Cómputo:
En cada ciclo computa 512 tokens de prompt + tokens de decode

Latencia TPOT Perfectamente Estable:
Ningún usuario percibe interrupciones ni parpadeos

Parámetro: --enable-chunked-prefill

Fuente: SARATHI: Efficient LLM Inference by Chunked-Prefills · Agrawal et al.

vLLM en Nube Privada OpenStack

Integración del motor de inferencia en máquinas virtuales KVM con GPU Passthrough y almacenamiento Ceph

PILA DE INFERENCIA EN NUBE PRIVADA OPENSTACK

Controlador Nova + Aceleración PCIe VFIO + Volúmenes Cinder Ceph para pesos de modelos

1. Aislamiento y Red OVN

Seguridad Perimetral:

- VM vLLM en red de proyecto aislada
- Grupos de seguridad filtran puerto 8000
- Acceso interno sin salir a internet

Soberanía de Datos:

Los prompts industriales nunca abandonan el centro de datos local.

2. Aceleración Directa VFIO

Passthrough PCI Completo:

- Nova asigna la tarjeta física entera
- Driver NVIDIA nativo dentro de la VM
- Cero sobrecarga de hipervisor (< 0.5 %)

Throughput Idéntico:

Mismo rendimiento que bare-metal con las ventajas de gestión de la nube.

3. Repositorio de Pesos en Ceph

Almacén Persistente Cinder:

- Modelos descargados en pool Ceph
- Clonado CoW instantáneo para réplicas
- Lectura de alta velocidad en 100 GbE

Arranque Acelerado:

Levanta nuevos workers de inferencia en segundos compartiendo los pesos base.

Fuente: OpenStack AI/ML Reference Architecture · OpenInfra Foundation

Recuperación de Fallos y Errores XID

Detección automatizada de anomalías en hardware y sustitución autónoma de pods de inferencia

1. Monitoreo de Kernel XID

Mensajes del Driver en dmesg:

NVRM: Xid (PCI:0000:01:00): 31, ...

Códigos de Falla Críticos:

- XID 31: Violación de página de memoria
- XID 79: Caída de tarjeta del bus PCIe

Detección Inmediata:

El daemon NVIDIA Node Problem Detector captura el evento en milisegundos.

2. Drenado y Aislamiento

Cordon & Drain en Kubernetes:

kubectl cordon nodo-gpu-03

Protección del Tráfico en Vuelo:

El balanceador retira el pod fallido del pool de endpoints sin cortar respuestas

Reintentos Idempotentes:

Las peticiones abortadas se reenvían de inmediato a un pod saludable.

3. Auto-Reparación y Reinicio

Reset de Bus PCIe por Software:

nvidia-smi --gpu-reset

Recuperación de Pods en Reserva:

ReplicaSet levanta un pod sustituto en nodo alternativo en menos de 60s

Disponibilidad 99.9 %:

El servicio empresarial continúa sin degradación perceptible para usuarios.

Fuente: NVIDIA GPU Diagnostic and XID Errors Guide · NVIDIA Developer

GOBERNANZA FINANCIERA

Auditoría de Tokens y FinOps en IA

Mecanismos de imputación de costos (Chargeback / Showback) para controlar el consumo entre departamentos

1. Identificación por API Key

Cabeceras de Rastreo:
Authorization: Bearer cic-planta-key
Asocia cada llamada a un centro de costos

Límites de Tasa (Rate Limiting):
Cuotas de tokens por minuto (TPM)
Evita que un script sature el clúster

Middleware Kong / Envoy:
Inspecciona y valida cuotas
antes de admitir peticiones en vLLM

Control estricto de acceso

2. Conteo Exacto de Tokens

Desglose de Facturación:
prompt_tokens vs completion_tokens
Registro inmutable en logs JSON

Telemetría OpenTelemetry:
Spans distribuidos por solicitud
Mide duración y costo por inferencia

Herramienta: LiteLLM Proxy
Base de datos de auditoría
y tarificación virtual por área

Precisión analítica total

3. Imputación de Costos

Modelo Showback (Transparencia):
Reporta el consumo teórico a cada jefe
sin transferir dinero presupuestario

Modelo Chargeback (Facturación):
Debita el costo de la infraestructura
del presupuesto del departamento

Incentivo de Eficiencia:
Fomenta que los desarrolladores usen
prompts concisos y modelos pequeños

Sostenibilidad financiera corporativa

Fuente: FinOps Foundation Guidelines for AI and Generative Workloads (2026)

CASO DE ESTUDIO INDUSTRIAL

Despliegue vLLM en Planta Barranquilla

Topología de inferencia local con Llama 3 8B AWQ sobre 2 GPUs RTX 6000 Ada en la CIC

1. Servidor de Cómputo Local

Chasis Físico Dell R750:

2x Intel Xeon Gold (64 núcleos)
256 GB RAM ECC + 2x 25 GbE LACP

Aceleradores de Vídeo:

2x NVIDIA RTX 6000 Ada (48 GB c/u)
Total VRAM: 96 GB a 960 GB/s de ancho

Entorno OpenStack:

VM Ubuntu 24.04 con passthrough
VFIO nativo sin pérdida de cómputo.

2. Motor vLLM Configurado

Modelo Seleccionado:

Llama 3 8B Instruct cuantizado en AWQ
Ocupación de pesos: Apenas 5.5 GB VRAM

Parámetros de Arranque:

```
--tensor-parallel-size 2
--kv-cache-dtype fp8 --max-model-len 8192
```

Capacidad Operativa:

Soporta más de 80 operadores de planta concurrentes con TTFT < 80 ms.

3. Resultados en Planta

Latencia de Red Mínima:

< 4 ms desde terminales de planta
Cero dependencia de enlaces de internet

Throughput Medido:

185 tokens/segundo por usuario activo
Fluidez instantánea de lectura

Seguridad Total:

Telemetría de reactores químicos
100 % aislada bajo estándar ISA-95.

CRITERIOS DE INGENIERÍA

Decálogo de Operación Self-Hosted

Directrices obligatorias para la estabilidad, escalabilidad y eficiencia de motores de inferencia

Reglas 1 a 5: Memoria y Rendimiento

1. PagedAttention Obligatorio en VRAM
Descartar motores que exijan memoria contigua para erradicar el desperdicio.

2. Activar Continuous Batching
Planificar a nivel de token para eliminar el padding inútil y reducir la cola.

3. Cuantizar la KV-Cache en FP8
Duplicar concurrencia con AWQ e INT8 KV.

4. Implementar Chunked Prefill
Dividir prompts extensos para proteger el percentil P99 de latencia interactiva.

5. Decodificación Especulativa en Cargas Críticas
Integrar modelos borrador ligeros.

Reglas 6 a 10: Operación y Gobernanza

6. Limitar Tensor Parallelism al Mismo Host
Reservar TP para tarjetas unidas por NVLink; no cruzar redes WAN o LAN.

7. Autoescalado con KEDA por Tokens en Espera
Escalar réplicas basado en peticiones encoladas y no en CPU genérica.

8. Detección Temprana de Errores XID
Monitorear dmesg para aislar nodos con fallas de hardware antes del colapso.

9. Control de Cuotas por API Key y Pasarela
Imponer límites de tokens por minuto (TPM) para prevenir abuso interno.

10. Auditoría de FinOps y Chargeback
Imputar costos de cómputo y energía.

Fuente: Production LLM Serving Best Practices · vLLM & MLPerf Engineering Guidelines

SEMANA 15 · ARQUITECTURA EN LA NUBE

IA en la Nube II: GPU, Cuantización y Servicios Gestionados frente a Auto-Alojados

Parte 4 de 4 · Servicios Gestionados frente a Auto-Alojados: Arquitectura, TCO y Gobierno

Ing. Rodolfo José Cañas Cervantes
Universidad de la Costa (CUC) · Ingeniería de Sistemas

Arquitectura en la Nube · Periodo 2026-2 · Barranquilla, Colombia

El Dilema: Gestionado vs Auto-Alojado

Compensación entre velocidad de entrega, costo operativo, latencia y control de soberanía

Modelo como Servicio Gestionado (MaaS)

Ventajas Clave:
Cero servidores · Escalado elástico · Modelos frontera

Riesgos y Limitaciones:
Costo variable alto a escala · Límites TPM/RPM

Perfil de Adopción Ideal:
Prototipado ágil, tráfico esporádico o variado, startups y MVPs iniciales

Facturación:
Pago por token consumido (Entrada + Salida)

Infraestructura Auto-Alojada (Self-Hosted)

Ventajas Clave:
Control de pesos · Baja latencia · Soberanía

Riesgos y Limitaciones:
Costo fijo por GPU ociosa · Mantenimiento 24/7

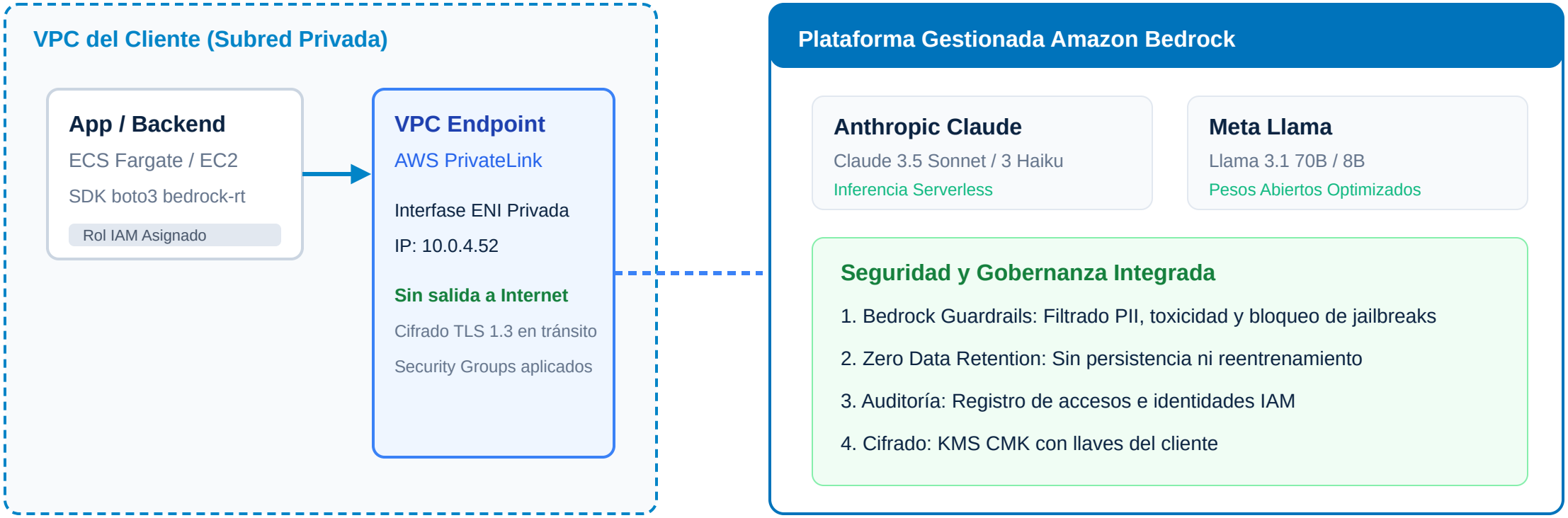
Perfil de Adopción Ideal:
Alto volumen continuo o secreto industrial

Facturación:
Costo fijo por hora de cómputo GPU activa

Fuente: AWS Prescriptive Guidance: GenAI Deployment Patterns · AWS Architecture Center

Arquitectura de AWS Bedrock

Invocación sin servidor, aislamiento en VPC privada y autenticación unificada IAM



Fuente: Amazon Bedrock User Guide: Security and Network Isolation · AWS Documentation

On-Demand vs Capacidad Aprovisionada

Diferencias operativas entre facturación por token y reserva de unidades de modelo dedicadas

Modalidad Bajo Demanda (On-Demand)

Métrica de Cobro:
Tarifa por cada 1,000 tokens procesados (Entrada / Salida)

Disponibilidad de Capacidad:
Compartida multinquilino · Sujeta a límites de cuota regional

Comportamiento en Picos:
Riesgo de HTTP 429 (Too Many Requests) si se satura el TPM regional

Compromiso Financiero:
Cero compromiso temporal · Ideal para tráfico impredecible

Capacidad Aprovisionada (Provisioned Throughput)

Métrica de Cobro:
Tarifa horaria por Unidad de Modelo (Model Unit - MU)

Garantía de Rendimiento:
Throughput y latencia garantizados · Cero rechazos por 429

Modelos Personalizados:
Requisito obligatorio para desplegar adaptadores LoRA fine-tuneados

Compromiso Financiero:
Contrato sin compromiso, 1 mes o 6 meses · Costo fijo continuo

Fuente: Amazon Bedrock Pricing & Provisioned Throughput · AWS Documentation

Aislamiento y Retención Cero de Datos (ZDR)

Garantías contractuales y técnicas contra el entrenamiento no autorizado con datos corporativos

1. Cero Entrenamiento

Garantía MaaS:

Ni AWS ni proveedores terceros usan prompts para reentrenar modelos.

Protección contractual contra fugas de propiedad intelectual corporativa.

2. Cero Persistencia (ZDR)

Memoria Volátil:

Prompts e inferencias se procesan en RAM GPU y se destruyen al finalizar.

No quedan copias en logs del proveedor a menos que el cliente lo configure en S3.

3. Aislamiento Criptográfico

Cifrado de Extremo a Fin:

TLS 1.3 en tránsito con perfect forward secrecy. KMS CMK para almacenamiento.

Claves gestionadas en módulos HSM validados bajo estándar FIPS 140-3.

4. Cumplimiento Legal

Certificaciones:

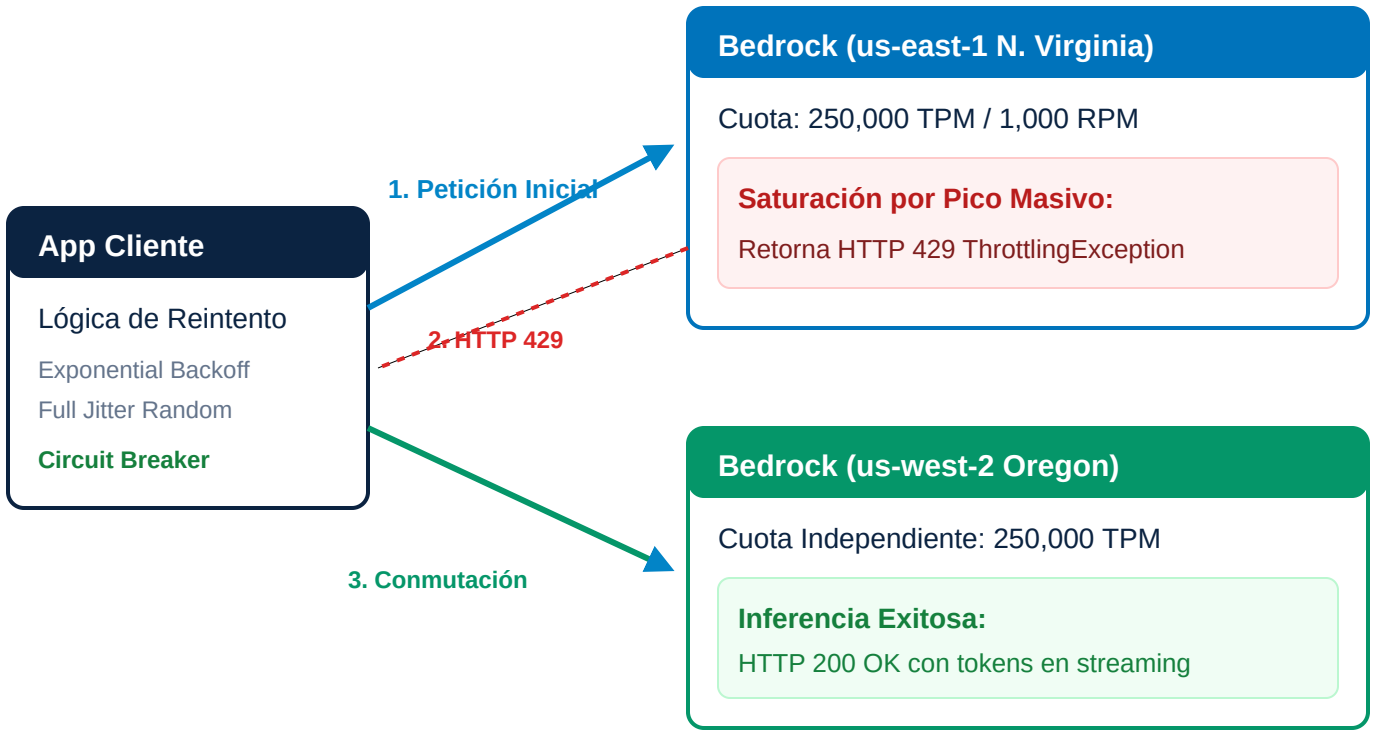
SOC 1/2/3 · ISO 27001 HIPAA eligible · GDPR Ley 1581 (Colombia).

Términos contractuales vinculantes respaldados por acuerdos de procesamiento.

Fuente: AWS Customer Agreement & Generative AI Data Protection Policy · AWS Trust Center

Gestión de Cuotas y Límites de Tasa

Mitigación del error HTTP 429 con retroceso exponencial, jitter y conmutación entre regiones



Estrategias Anti-Throttling

1. Retroceso Exponencial con Jitter:
Evita el problema de la manada dispersando reintentos con desfase aleatorio.

2. Ruteo Multi-Región Activo:
Las cuotas de Bedrock son por región. Alternar entre regiones duplica el tope efectivo.

3. Solicitud de Aumento de Cuota:
Service Quotas console antes de lanzar campañas masivas en producción.

Fuente: AWS Architecture Blog: Handling Generative AI API Throttling & Multi-Region Fallback

Responsabilidad Compartida en IA

Límites de custodia entre el proveedor de nube y el cliente según el modelo de servicio adoptado

IaaS (EC2 / OpenStack)	PaaS (SageMaker Endpoints)	MaaS (Bedrock Serverless)
Datos y Prompts del Negocio (Cliente)	Datos y Prompts del Negocio (Cliente)	Datos, Prompts y Salidas (Cliente)
Pesos del Modelo y Cuantización (Cliente)	Selección de Modelo y Fine-Tuning (Cliente)	Configuración de Guardrails (Cliente)
Motor vLLM, CUDA y Drivers (Cliente)	Contenedor DLC e Inferencia (Nube)	Pesos del Modelo y Actualizaciones (Nube)
Sistema Operativo y Parcheo (Cliente)	Autoescalado y Parcheo SO (Nube)	Motor de Ejecución y Escalabilidad (Nube)
Hardware Físico e Hipervisor (Nube)	Infraestructura GPU Subyacente (Nube)	Clúster de Cómputo y Silicio GPU (Nube)

Fuente: AWS Shared Responsibility Model for Generative AI · AWS Security Whitepaper

Estructura de Precios por Millón de Tokens

Costos reales de entrada frente a salida en modelos fundacionales líderes del mercado

Modelo Fundacional	Proveedor	Tokens Entrada (1M)	Tokens Salida (1M)	Relación
Claude 3.5 Sonnet Frontier multimodal	Anthropic	**$3.00 USD**	**$15.00 USD**	1 : 5
Claude 3 Haiku Alta velocidad y bajo costo	Anthropic	**$0.25 USD**	**$1.25 USD**	1 : 5
Llama 3.1 70B Pesos abiertos gestionados	Meta	**$0.72 USD**	**$0.72 USD**	1 : 1
Llama 3.1 8B Modelo liviano para tareas ágiles	Meta	**$0.22 USD**	**$0.22 USD**	1 : 1

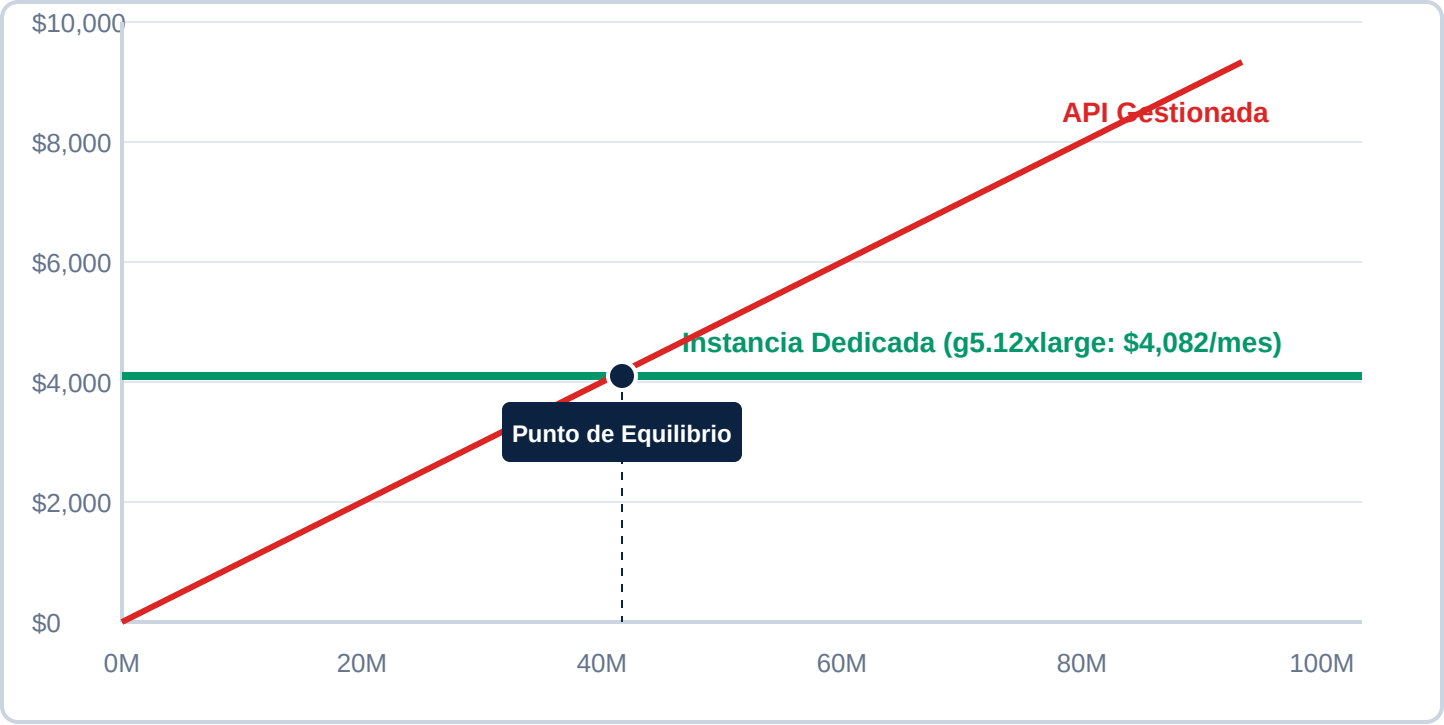
Asimetría de Costos:

Los tokens de salida son 5 veces más caros en modelos propietarios debido a la fase de decodificación autorregresiva secuencial.

Fuente: AWS Bedrock Pricing Table (us-east-1) · Consulta oficial 2026-11

Punto de Equilibrio: Gestionado vs GPU

Curvas de costo mensual según volumen diario de inferencia (Tokens/día vs Instancia EC2 g5.12xlarge)



Criterio Financiero

Bajo Volumen (< 40M/día):

La API gestionada es más barata.

Se evitan $4,000+ USD mensuales de alquiler de GPU estática ociosa.

Alto Volumen (> 40M/día):

Auto-alojar es más rentable.

La GPU opera a plena capacidad y el costo marginal por token es ~0.

Nota de Operación:

Sumar costos de ingeniería y soporte 24/7 al presupuesto de infraestructura propia.

Fuente: Analysis of GenAI TCO: Break-Even Modeling for LLM Inference · Cloud Architecture Studies

Perfil de Latencia y SLAs

Comparativa de tiempo al primer token (TTFT) y variabilidad P50/P99 en redes locales vs públicas

Tiempo al Primer Token (TTFT) en Escenarios Típicos

vLLM Local en Red LAN (Nodo Propio):



~85 ms (P50) / 120 ms (P99)

vLLM en EC2 (Misma VPC y Subred):



~140 ms (P50) / 210 ms (P99)

AWS Bedrock (vía PrivateLink):



~450 ms (P50) / 950 ms (P99)

API Pública a través de Internet (TLS Wan):



~850 ms (P50) / hasta 2,800 ms (P99 en hora pico)

Impacto en la Experiencia de Usuario

Casos Críticos de Tiempo Real:

Autocompletado de código, asistentes de voz interactivos y sistemas SCADA requieren TTFT inferior a 200 ms.

Casos Asíncronos y por Lotes:

Resúmenes nocturnos de correo, análisis documental RAG y traducción toleran latencias de 1 a 3 segundos sin impacto.

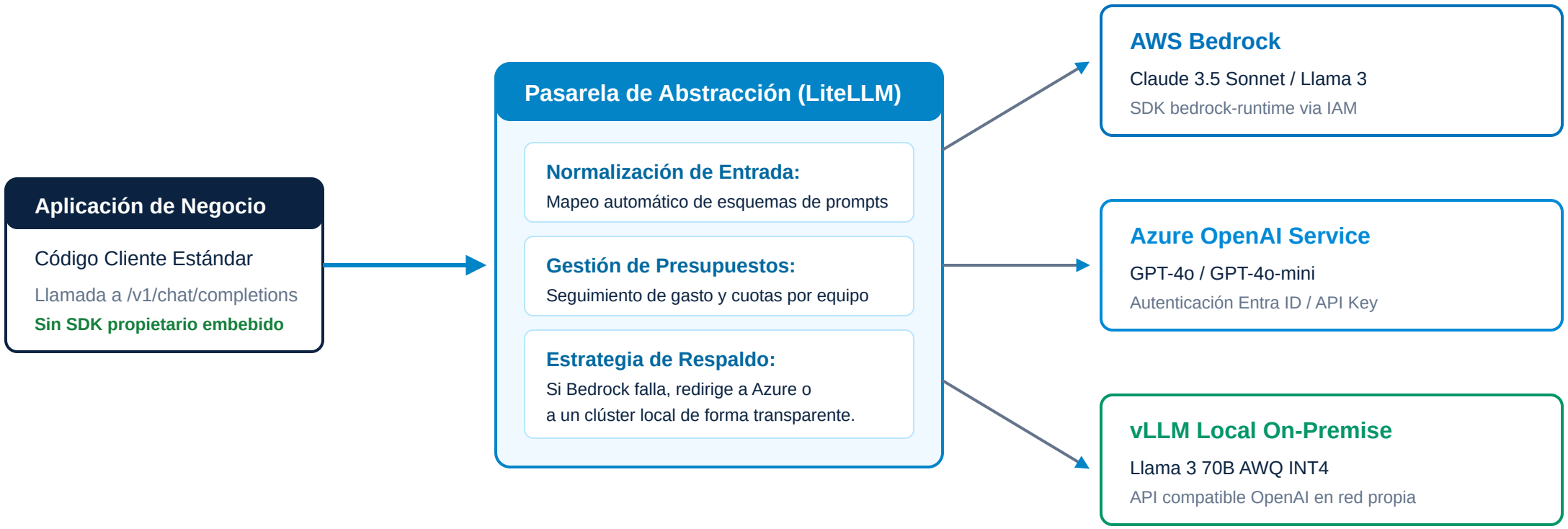
Variabilidad en la Cola (P99):

Las APIs compartidas sufren fluctuaciones por tráfico de terceros; el motor auto-alojado asegura un SLA consistente y predecible.

Fuente: Benchmarking LLM Serving Systems: TTFT and Inter-Token Latency · MLPerf Inference

Mitigación del Vendor Lock-in

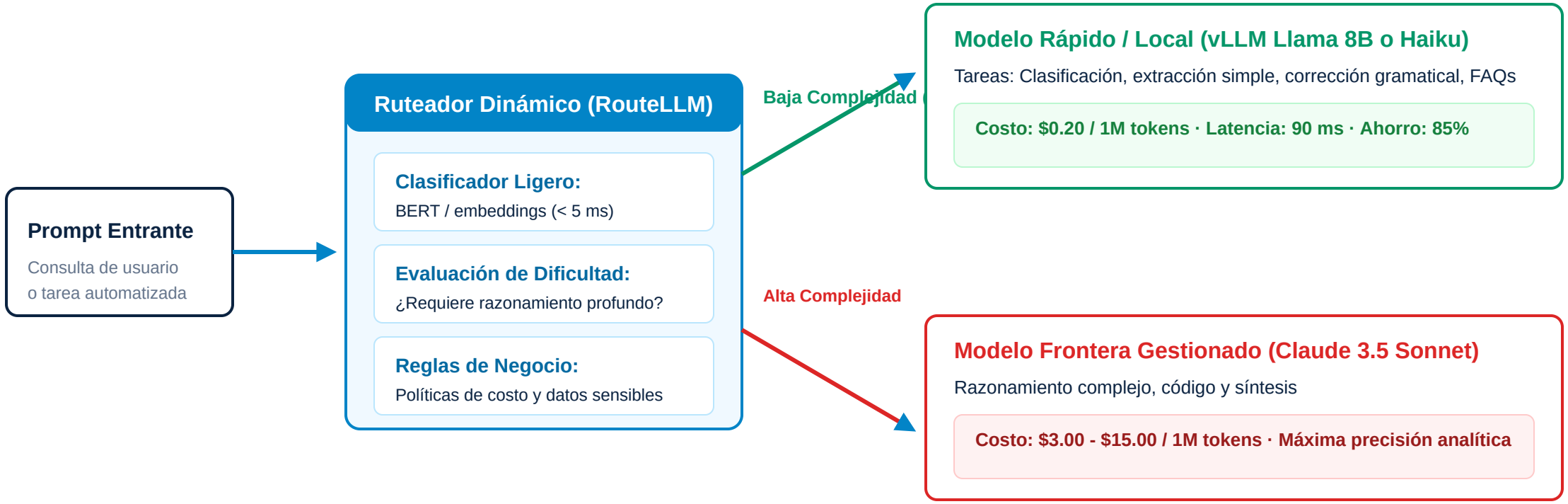
Capas de abstracción y adopción de la especificación OpenAI API para desacoplar el código del proveedor



Fuente: The OpenAI API Reference & LiteLLM Multi-Provider Proxy Documentation

Ruteo Dinámico Inteligente de Inferencia

Clasificación previa de la complejidad del prompt para alternar entre modelos pequeños y modelos frontera



Fuente: RouteLLM: Learning to Route LLMs with Preference Data · LMSYS Org & UC Berkeley

Soberanía de Datos en Colombia

Cumplimiento de la Ley 1581 de 2012 y directrices de la Superintendencia de Industria y Comercio (SIC)

1. Ley 1581 de 2012 (Habeas Data)

Transferencia Internacional:

Prohibida hacia países que no cuenten con niveles adecuados de protección de datos.

Consentimiento Expreso:

Obligatorio informar al titular que sus datos serán procesados mediante modelos de IA.

Principio de Finalidad:

Los datos recolectados no pueden emplearse para fines no pactados en la política de privacidad.

2. Directrices SIC para IA

Privacidad desde el Diseño:

Anonimización y enmascaramiento de datos sensibles antes de enviar prompts a la nube.

Transparencia y No Discriminación:

Auditoría sobre sesgos algorítmicos en decisiones automatizadas de crédito o contratación.

Seguridad Demostrable:

Exigencia de cifrado robusto y acuerdos contractuales de no retención con proveedores.

3. Solución Arquitectónica

Enmascaramiento Previo:

Módulo regex / spaCy sustituye nombres, cédulas y montos por tokens ficticios <ID_1>.

Residencia Local Obligatoria:

Historias clínicas e inteligencia judicial procesadas exclusivamente en vLLM local.

Bitácora de Cumplimiento:

Registro inmutable en S3 Object Lock para evidenciar conformidad ante requerimientos SIC.

Fuente: Ley Estatutaria 1581 de 2012 y Guía para el Tratamiento de Datos Personales en IA · SIC Colombia

SÍNTESIS METODOLÓGICA

Matriz de Decisión Arquitectónica

Cuadro comparativo multidimensional para seleccionar el modelo operativo según los requisitos del sistema

Dimensión de Evaluación	Servicio Gestionado (Bedrock)	Auto-Alojado Nube (EC2 vLLM)	Auto-Alojado On-Premise
Tiempo de Salida al Mercado	**Inmediato (Horas / Días)**	Medio (1 - 2 Semanas)	Lento (Meses por Hardware)
Estructura de Costos	100% OpEx variable por token	OpEx fijo por hora de cómputo	CapEx inicial elevado + OpEx
Privacidad y Soberanía	Contractual (ZDR en nube pública)	Aislamiento en VPC propia	**Control Físico Absoluto (Aire)**
Personalización de Pesos	Limitada a LoRA / Adaptadores	Completa (Cualquier arquitectura)	Completa (Cualquier arquitectura)
Carga Operativa del Equipo	**Mínima (Cero administración)**	Media (K8s, vLLM, parches)	Alta (Hardware, refrigeración, red)
Recomendación Típica	Prototipos, MVPs y tráfico variable	Cargas estables de mediano/alto volumen	Banca, salud o secreto industrial

Fuente: Enterprise Architecture Decision Framework for Generative AI Workloads

Conclusiones y Conexión con Semana 16

De la infraestructura y optimización del silicio hacia las capacidades cognitivas avanzadas de los modelos

Lo Dominado en Semana 15 (Infraestructura de IA)

1. Silicio Acelerador y GPUs:
Ancho de banda HBM3, Tensor Cores y cuello de botella de memoria.

2. Cuantización Rigurosa:
Compresión matemática AWQ y GPTQ a 4 bits protegiendo la perplejidad.

3. Motores de Inferencia vLLM:
Algoritmo PagedAttention y batching continuo para exprimir la VRAM.

4. Estrategia Económica y Legal:
Punto de equilibrio en costos y cumplimiento de la Ley 1581 colombiana.

Hacia la Semana 16: Capacidades Cognitivas y Cierre

1. Razonamiento y Planificación:
Cadena de pensamiento, árboles de búsqueda y modelos reflexivos.

2. Sistemas Agénticos y Delegación:
Uso autónomo de herramientas, llamadas a funciones y protocolos MCP.

3. Ventanas Masivas de Contexto:
Arquitecturas de dos millones de tokens frente a sistemas RAG externos.

4. Repaso Integral de la Unidad 3:
Consolidación OpenStack, inferencia y gobierno.